

Principles of Machine Learning

Week 3: Classification Foundations & Baseline Models

Victor Wandera Lumumba, MSc.

Statistician | Data Analyst | ML Expert

Mediacrest Training College

13th Oct 2026

Abstract

Welcome to Week 3! Today, we transition from data preprocessing to the core of predictive modeling: **Machine Learning**. We will explore the foundational principles of ML, dive deep into baseline classification algorithms, understand their mathematical underpinnings, and learn how to rigorously evaluate and report model performance using a real-world clinical cohort (Hypertension).

- **ML Fundamentals:** Definitions, paradigms (Supervised, Unsupervised, RL), and real-world applications.
- **Classification Models:** Deep dive into Logistic Regression, KNN, SVM, and Naive Bayes.
- **Mathematical Foundations:** The equations and optimization objectives driving these algorithms.
- **Evaluation & Reporting:** Confusion matrices, ROC-AUC, and clinical interpretation of results.

Session Objectives

By the end of this session, learners will be able to:

- Define Machine Learning and distinguish between its core paradigms.
- Formulate a clinical or business problem as a supervised classification task.
- Explain the mathematical mechanics of Logistic Regression, KNN, SVM, and Naive Bayes.
- Select appropriate evaluation metrics based on class imbalance and clinical costs.
- Interpret model outputs and generate a professional, stakeholder-ready performance report.
- Understand the trade-offs between model interpretability and predictive power.

Part 1: Introduction to Machine Learning

"From Explicit Rules to Learned Patterns"

What is Machine Learning?

- **Arthur Samuel (1959):** "Field of study that gives computers the ability to learn without being explicitly programmed."
- **Tom Mitchell (1997):** "A computer program is said to *learn* from experience E with respect to some task T and some performance measure P, if its performance on T, as measured by P, improves with experience E."
- Instead of writing rules to solve a problem, we feed data to an algorithm, and it **discovers the rules** (patterns) itself.

The Paradigm Shift

Traditional Programming: `Data + Rules = Answers`
Machine Learning: `Data + Answers = Rules`

Types of Machine Learning

1. Supervised Learning

- Data has labels (targets).
- *Tasks:* Classification, Regression.
- *Example:* Predicting HTN status.

2. Unsupervised Learning

- Data has no labels.
- *Tasks:* Clustering, Dimensionality Reduction.
- *Example:* Grouping patients by biomarkers.

3. Reinforcement Learning

- Agent learns via rewards/penalties.
- *Tasks:* Decision making, control.
- *Example:* Treatment dosage optimization.

4. Semi-Supervised

- Mix of labeled and unlabeled data.
- Useful when labeling is expensive.

Supervised Learning Deep Dive

- The most common paradigm in business and healthcare AI.
- We provide the model with a feature matrix X and a target vector y.
- The goal is to learn a mapping function $f : X \rightarrow y$.

Regression

Predicting a **continuous** number.

Examples:

- Predicting blood pressure (mmHg).
- Forecasting monthly sales (KES).

Classification

Predicting a **discrete** category.

Examples:

- HTN: Yes or No.
- Tumor: Benign or Malignant.

Applications of ML

- **Healthcare:** Disease diagnosis (HTN, Diabetes), patient readmission prediction, medical imaging analysis.
- **Finance:** Credit scoring, algorithmic trading, fraud detection.
- **Retail & E-commerce:** Customer churn prediction, recommendation systems, inventory forecasting.
- **Telecommunications:** Network optimization, predictive maintenance.

The Common Thread

In all these domains, ML is used to **automate decision-making**, **uncover hidden patterns**, and **predict future outcomes** at scale.

The ML Pipeline

Building a model is just one step in a larger workflow:

1. **Problem Formulation:** Define the business/clinical question.
2. **Data Collection & Cleaning:** (Weeks 1 & 2) Handle missing values, outliers.
3. **Exploratory Data Analysis (EDA):** Understand distributions and relationships.
4. **Feature Engineering:** Create predictive variables.
5. **Model Selection & Training:** (Today) Fit algorithms to data.
6. **Evaluation:** Measure performance on unseen data.
7. **Deployment & Monitoring:** Put the model into production.

Part 2: Classification Foundations

"Predicting Categories in the HTN Cohort"

The Classification Problem

- **Goal:** Assign an input observation to one of two or more predefined classes.
- **Binary Classification:** Only two possible outcomes (e.g., HTN vs. No HTN, Spam vs. Not Spam).
- **Multiclass Classification:** More than two outcomes (e.g., predicting the species of an iris flower).
- **Output:** Models typically output a **probability** $P(y = 1|X)$, which is then thresholded (usually at 0.5) to make a hard class prediction.

Context: The Hypertension (HTN) Cohort

Our Use Case

Predicting the presence of Hypertension in an HIV-positive cohort based on clinical and demographic markers.

- **Target Variable (y):** `HTN` (1 if SBP $\geq$ 140 or DBP $\geq$ 90, else 0).
- **Features (X):** age, `BMI`, `log_creat_centered`, `hgb_centered`, `male.gender`, etc.
- **Challenge:** The dataset is **imbalanced** (fewer HTN cases than non-HTN), which impacts how we evaluate our models.

Baseline Models Overview

Before deploying complex Deep Learning or Ensemble models (like XGBoost), we establish a baseline using fundamental algorithms:

- **Logistic Regression:** Linear, highly interpretable.
- **K-Nearest Neighbors (KNN):** Instance-based, non-parametric.
- **Support Vector Machines (SVM):** Margin-based, powerful in high dimensions.
- **Naive Bayes:** Probabilistic, fast, assumes feature independence.

These models provide a benchmark. If a complex model cannot beat Logistic Regression, the complex model is likely overfitting or unnecessary.

Part 3: Model 1
Logistic Regression

"The Workhorse of Classification"

Logistic Regression: Concept

- Despite the name, it is a **classification** algorithm, not regression.
- It calculates a linear combination of input features, just like Linear Regression.
- However, it passes this linear output through a non-linear **Sigmoid (Logistic) function** to squash the output into a probability between 0 and 1.
- **Strengths:** Highly interpretable, fast to train, provides feature importance (odds ratios).

Logistic Regression: The Math

1. The Linear Predictor (Logit):

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

2. The Sigmoid Function (Probability):

$$P(y = 1|X) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Interpretation

If $P(y = 1|X) \geq 0.5$, we predict class 1 (HTN Present). The coefficients β_i represent the log-odds change for a 1-unit increase in x_i.

Logistic Regression: Loss Function

Why not use Mean Squared Error (MSE)? Because the sigmoid function makes the MSE loss surface **non-convex** (many local minima).

Instead, we use **Log-Loss (Binary Cross-Entropy)**:

$$L(\beta) = -\frac{1}{N}\sum_{i=1}^{N}\Big[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\Big]$$

- Penalizes confident wrong predictions heavily.
- Convex, ensuring Gradient Descent finds the global minimum.

Part 4: Model 2
K-Nearest Neighbors

"Tell me who your neighbors are..."

KNN: Concept

- KNN is a **lazy learner** (instance-based). It doesn't "learn" a mathematical model during training; it simply memorizes the dataset.
- **Prediction:** To classify a new point, find the K closest points in the training data. The majority class among those K neighbors wins.
- **Key Hyperparameter:** K (the number of neighbors).
  - Small K (e.g., 1): Noisy, overfits.
  - Large K: Smooth decision boundaries, may underfit.

KNN: The Math (Distance Metrics)

"Closeness" is defined by distance metrics. For continuous features, we typically use **Euclidean Distance**:

$$d(p, q) = \sqrt{\sum_{i=1}^{n} (p_i - q_i)^2}$$

Other metrics include:

- **Manhattan (L_1):** $d(p, q) = \sum_{i=1}^{n} |p_i - q_i|$ (Robust to outliers).
- **Minkowski (L_p):** Generalization of both.

Crucial Step

KNN is **highly sensitive to feature scales**. You *must* standardize (e.g., Z-score) your numerical features before training!

Part 5: Model 3
Support Vector Machines

"Finding the Widest Street"

SVM: Concept

- SVM aims to find the optimal **hyperplane** that separates the classes with the maximum **margin**.
- The **margin** is the "street" width between the closest data points of each class.
- The data points lying exactly on the edge of the street are called **Support Vectors**. They "support" or define the boundary.
- **Strengths:** Effective in high-dimensional spaces, memory efficient (only uses support vectors).

SVM: The Math (Optimization)

Hard Margin SVM (Linearly Separable):

$$\min_{w,b} \frac{1}{2}||w||^2 \quad \text{subject to} \quad y_i(w \cdot x_i + b) \geq 1 \ \ \forall i$$

Soft Margin SVM (Real-world data with overlap): Introduces slack variables ξ_i and a regularization parameter C:

$$\min_{w,b,\xi} \frac{1}{2}||w||^2 + C\sum_{i=1}^{N} \xi_i$$

- **High C:** Strict margin, penalizes misclassifications heavily (Risk of overfitting).
- **Low C:** Wider margin, allows some misclassifications (Better generalization).

SVM: The Kernel Trick

What if the data is not linearly separable in the original space?

- SVM maps the data into a **higher-dimensional space** where a linear hyperplane can separate it.
- Computing this mapping explicitly is computationally expensive. The **Kernel Trick** computes the dot product in the high-dimensional space directly, without ever visiting it.

Common Kernels

- **Linear:** $K(x, y) = x \cdot y$
- **Polynomial:** $K(x, y) = (\gamma x \cdot y + r)^d$
- **RBF (Gaussian):** $K(x, y) = \exp(-\gamma ||x - y||^2)$ (Most popular)

Part 6: Model 4
Naive Bayes

"Probabilities and Independence"

Naive Bayes: Concept

- A probabilistic classifier based on **Bayes' Theorem**.
- It calculates the probability of each class given the input features.
- **The "Naive" Assumption:** It assumes that all features are **conditionally independent** of each other given the class label.
- *Example:* It assumes a patient's Age and BMI don't influence each other when predicting HTN. (This is rarely true in reality, hence "naive", but the model still performs surprisingly well).

Naive Bayes: The Math

Bayes' Theorem:

$$P(C_k|X) = \frac{P(C_k)P(X|C_k)}{P(X)}$$

Applying the **Naive Independence Assumption**:

$$P(X|C_k) = P(x_1|C_k) \times P(x_2|C_k) \times \cdots \times P(x_n|C_k)$$

Final Prediction Rule:

$$\hat{y} = \arg\max_{C_k} P(C_k) \prod_{i=1}^{n} P(x_i|C_k)$$

We simply multiply the prior probability of the class by the likelihood of each feature, and pick the class with the highest resulting score.

Part 7: Model Evaluation & Metrics

"Beyond Simple Accuracy"

The Confusion Matrix

The foundation of classification evaluation. It cross-tabulates Actual vs. Predicted classes.

		Predicted	
	Actual	**Positive (1)**	**Negative (0)**
2*	**Positive (1)**	True Positive (TP)	False Negative (FN)
	Negative (0)	False Positive (FP)	True Negative (TN)

- **FP (Type I Error):** False alarm (Predicted HTN, but healthy).
- **FN (Type II Error):** Missed case (Predicted healthy, but has HTN). **High clinical risk!**

Precision, Recall, and F1-Score

Precision: Out of all predicted positives, how many were actually positive?

$$\text{Precision} = \frac{TP}{TP + FP}$$

Focus: Minimizing False Positives.

Recall (Sensitivity): Out of all actual positives, how many did we find?

$$\text{Recall} = \frac{TP}{TP + FN}$$

Focus: Minimizing False Negatives.

F1-Score: The harmonic mean of Precision and Recall.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

The Trade-off

You can easily get 100% Recall by predicting *everyone* as positive, but Precision will plummet. F1 balances both.

ROC-AUC and PR-AUC

Threshold-independent metrics that evaluate the model's ability to **rank** predictions.

- **ROC Curve:** Plots Recall (TPR) vs. FPR across all thresholds.
- **ROC-AUC:** Area Under the ROC Curve. (0.5 = random guessing, 1.0 = perfect). Good for balanced datasets.
- **PR Curve:** Plots Precision vs. Recall.
- **PR-AUC:** Area Under the PR Curve. **Crucial for imbalanced datasets** (like our HTN cohort) because it focuses on the minority class and ignores True Negatives.

Part 8: Reporting Results

"Translating Math into Clinical Insights"

Results: Model Comparison (HTN Cohort)

Performance of baseline models on the held-out test set (Stratified 5-Fold CV for training):

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Reg.	0.845	0.780	0.710	0.743	0.821
KNN (K=7)	0.790	0.650	0.680	0.664	0.755
SVM (RBF)	**0.860**	**0.810**	0.740	**0.773**	**0.855**
Naive Bayes	0.810	0.700	0.650	0.674	0.780

Key Takeaway

SVM achieved the highest overall discrimination (ROC-AUC = 0.855) and F1-Score, handling the non-linear relationships between BMI, Age, and Creatinine best.

Results: Clinical Interpretation

How do we choose the "best" model for deployment?

- **If the goal is Screening (Maximize Recall):** We might tune the SVM or Logistic Regression threshold to achieve $> 90\%$ Recall. We accept more False Positives to ensure no hypertensive patient is missed (minimizing FN).
- **If the goal is Resource Allocation (Maximize Precision):** We want to be sure that when we flag a patient, they truly need intervention. We optimize for Precision to avoid alert fatigue.
- **Interpretability:** Logistic Regression allows us to extract **Odds Ratios**. We can tell clinicians exactly *how* a 1-unit increase in BMI changes the odds of HTN. SVM and KNN are "black boxes".

Summary & Best Practices

- **Start Simple:** Always establish a baseline with Logistic Regression before moving to SVM or Ensembles.
- **Preprocessing Matters:** KNN and SVM require scaled data; LR and NB are more robust (though scaling helps LR converge faster).
- **Context Drives Metrics:** Don't just optimize for Accuracy. In clinical imbalanced data, prioritize PR-AUC, Recall, or F1 based on the cost of False Negatives.
- **Document Everything:** The math, the hyperparameters, and the clinical trade-offs must be documented for auditability.

Next Steps: Week 3 Day 2

Moving to Ensemble Methods

- Today we built **baseline** models. Tomorrow, we will build **champion** models.
- **Decision Trees:** Learning to split data based on information gain.
- **Random Forest:** Bagging multiple trees to reduce variance.
- **XGBoost:** Boosting trees sequentially to correct errors.
- **SHAP & LIME:** Opening the "black box" to explain complex model predictions to clinicians.

Recommended Reading

Textbooks & Resources

- **An Introduction to Statistical Learning** (James, Witten, Hastie, Tibshirani) - Chapters 2 & 4.
- **Pattern Recognition and Machine Learning** (Christopher Bishop) - For deep mathematical intuition.
- **Scikit-Learn Documentation:** Excellent mathematical breakdowns of every algorithm.
- **Kaggle Learn:** "Intro to Machine Learning" and "Intermediate Machine Learning" micro-courses.

Common Pitfalls to Avoid

1. Data Leakage

Scaling data *before* splitting into train/test. The test set "leaks" into the training process. *Always use Pipelines!*

2. Over-reliance on Accuracy

In a dataset with 95% negative cases, a model that predicts "Negative" for everyone gets 95% Accuracy but 0% Recall. *Use Confusion Matrices and PR-AUC.*

3. Ignoring Class Weights

Failing to tell the model that the minority class (HTN) is more important. *Use* `class_weight='balanced'`.

Final Thoughts

The Golden Rule of ML

"A model is only as good as the data it was trained on, and the assumptions it was built upon."

- Math provides the engine.
- Data provides the fuel.
- Clinical/Business context provides the steering wheel.
- As data scientists, we must master all three to build AI that is not just accurate, but **trustworthy and actionable**.

Q&A and Wrap-up

Questions?

"In theory, there is no difference between theory and practice. In practice, there is."

Victor Wandera Lumumba
Statistician | Data Analyst | ML Expert
Mediacrest Training College